DSA0112-Object Oriented Programming in C++

ASSIGNMENT REPORT

HOTEL ACCOMMODATION AND RESERVATION MANAGEMENT USING C++

COURSE FACULTY: Dr. Sudha I

ABSTRACT

The Hotel Accommodation and Reservation Management System using C++ is a menu-driven, object-oriented application developed to manage essential hotel operations such as guest registration, room management, reservation processing, service management, cancellation, and billing. The system organizes hotel rooms into different categories, including Standard, Deluxe, and Suite rooms, and supports additional services such as food, laundry, and airport transportation. A guest can be registered with a unique Guest ID, after which the system allows room availability to be checked for a selected stay period before creating a reservation.

The application applies important C++ Object-Oriented Programming concepts such as classes and objects, encapsulation, abstraction, inheritance, hierarchical inheritance, multiple inheritance, virtual inheritance, pure virtual functions, function overriding, pointers, smart pointers, and runtime polymorphism. Virtual base classes are used to avoid duplication in multiple inheritance, while abstract classes define common behaviour for room and service categories. STL vectors are used for dynamic data storage, and exception handling is implemented to manage invalid Guest IDs, Room Numbers, service selections, and stay periods.

A key feature of the system is the reservation overlap checking mechanism. Instead of maintaining only a simple occupied or available status, the application compares the requested check-in and check-out period with existing active reservations. This helps prevent double booking while allowing the same room to be reserved for different non-overlapping periods. The billing module calculates accommodation charges based on the number of nights and room rate

and adds the cost of selected hotel services to generate the final bill. The proposed system provides a structured, reusable, and extensible solution for hotel reservation management while demonstrating the practical application of major C++ programming concepts.

1. INTRODUCTION

The hospitality industry depends heavily on efficient management of guests, room availability, reservations, services, cancellations, and billing. In a manually operated hotel environment, maintaining all these activities can become difficult when the number of guests and reservations increases. Manual booking methods can lead to problems such as double booking of rooms, incorrect calculation of accommodation charges, difficulty in retrieving reservation details, and improper management of additional hotel services. A computerized reservation system can overcome many of these difficulties by organizing information and automating repetitive operations.

The project titled **"Hotel Accommodation and Reservation Management Using C++"** is developed as a menu-driven console application that demonstrates the practical application of Object-Oriented Programming concepts in C++. The system manages hotel guests, different categories of rooms, reservations, additional hotel services, cancellation of reservations, and billing. The application provides separate room categories such as Standard, Deluxe, and Suite rooms. It also supports services such as food, laundry, and airport transportation.

The project has been designed not only to perform hotel reservation operations but also to demonstrate important C++ concepts such as classes and objects, encapsulation, abstraction, inheritance, hierarchical inheritance, multiple inheritance, virtual inheritance, pure virtual functions, function overriding, base-class pointers, smart pointers, runtime polymorphism, STL vectors, constructors, and exception handling. The overall design makes the system modular so that new room types or services can be introduced in the future without redesigning the complete application.

2. PROBLEM STATEMENT AND PROBLEM FORMULATION

The problem addressed in this project is the design and development of a menu-driven, object-oriented Hotel Reservation Management System using C++. The system must be capable of managing different room categories, guests, reservations, and hotel services. It should allow the hotel user to register a new guest, view the room catalog, identify rooms that are available for a requested stay period, make reservations, add additional services, cancel reservations, calculate the final bill, and display complete reservation information.

One of the important requirements of the problem is the use of inheritance to organize different types of rooms and services. The system is therefore designed with a common room hierarchy

and a separate service hierarchy. Abstract classes are introduced so that common operations such as calculating rates and displaying information are defined through common interfaces while their exact implementation is provided by the derived classes. Multiple inheritance and virtual inheritance are also incorporated to demonstrate the prevention of duplicated base-class data in a diamond inheritance structure.

The problem can be formulated in terms of three major stages: input, processing, and output. The input to the application includes guest information, room number, check-in and check-out days, reservation ID, selected service, service quantity, and menu choices. The system processes the information by validating the guest and room, examining existing reservations, identifying possible date conflicts, creating a reservation, attaching services, calculating room and service charges, and determining the final bill. The final output consists of guest registration confirmation, Guest ID, room availability information, Reservation ID, reservation status, selected services, accommodation charge, service charge, final bill, and relevant error messages.

A major consideration in the problem formulation is the handling of room availability. A simple Boolean value such as occupied = true or occupied = false is not sufficient for a reservation system because the same room may be occupied during one date range and available during another. Therefore, the proposed system determines availability by comparing the requested check-in and check-out days with the periods of previously created reservations. This approach allows future reservations to be managed correctly and prevents two guests from reserving the same room for overlapping periods.

3. OBJECTIVE AND EXPECTED OUTCOMES

The primary objective of this project is to develop a functional Hotel Accommodation and Reservation Management application using C++ while demonstrating the practical use of Object-Oriented Programming concepts. The system is intended to simplify the handling of guest information, room categories, reservations, additional services, cancellations, and billing within a single application.

Another important objective is to design the application in such a way that different hotel entities are represented using appropriate classes rather than implementing the complete application through unrelated functions. A common Room class is used to represent shared characteristics of hotel rooms, while StandardRoom, DeluxeRoom, and SuiteRoom are derived from it. Similarly, a common Service class represents hotel services, while specific service types are implemented as derived classes.

The project also aims to demonstrate abstraction by defining common operations through pure virtual functions. Runtime polymorphism is achieved through base-class pointers and smart pointers, enabling the program to handle different room and service objects dynamically. Virtual

inheritance is used to prevent duplication of the common base object when multiple inheritance is involved.

The expected result of the project is a working application that can register guests, assign unique Guest IDs, display different room categories, verify availability for a selected stay period, create reservations, prevent overlapping reservations, attach additional hotel services, cancel reservations, calculate bills automatically, and display complete reservation information. The system is also expected to handle invalid Guest IDs, Room Numbers, service selections, and incorrect stay periods using exception handling rather than terminating unexpectedly.

4. REQUIREMENTS, CONSTRAINTS AND ASSUMPTIONS

The project is developed using the C++ programming language and requires a compiler that supports at least C++17. The application can be implemented using an IDE such as Visual Studio Code or Code::Blocks and compiled using the GNU G++ compiler or MinGW compiler on Windows. Git is used for maintaining the source code, while GitHub is used for storing and submitting the complete working project files. The documentation can be prepared using Microsoft Word or another word-processing application before being converted into the required submission format.

The application does not require high-end hardware because it is a console-based academic project. A system with a dual-core processor, 4 GB of RAM, standard storage, keyboard, and monitor is sufficient for compiling and executing the application. Internet connectivity is required only for uploading the project files to the GitHub repository.

The current version of the system has certain constraints. It is a console-based program and does not include a graphical user interface. Information is maintained in memory only during the execution of the program, which means that the guest and reservation data are removed after the application is closed. A permanent database has not been included because the purpose of the assignment is primarily to demonstrate C++ Object-Oriented Programming concepts. Actual calendar dates are also simplified using numerical day values. Online payment, login authentication, room photographs, refund processing, and real-time online booking are outside the scope of the current implementation.

The system assumes that every guest is given a unique Guest ID and every reservation is given a unique Reservation ID. It is assumed that the check-out day is always greater than the check-in day. Each room category has a predefined price per night, and each additional hotel service has a fixed price. A cancelled reservation is considered inactive and therefore does not prevent another guest from booking the room during the same period. The demonstration hotel contains a small number of predefined rooms, although the architecture allows additional rooms to be included easily.

5. APPLICATION OF RELEVANT COURSE KNOWLEDGE

The design of the Hotel Accommodation and Reservation Management System applies the major Object-Oriented Programming concepts studied in C++.

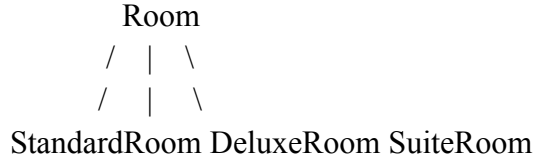
The system uses **classes and objects** to represent real-world hotel entities. The Guest class represents a hotel guest, the Reservation class represents booking information, the Room hierarchy represents different accommodation categories, and the Service hierarchy represents optional hotel facilities. A HotelSystem object controls the complete application and provides the menu operations.

Encapsulation is implemented by keeping important data members within their respective classes. For example, the Guest ID, guest name, and phone number are maintained inside the Guest class. These details are accessed through member functions instead of being freely modified from outside the class. The same design principle is used in the Reservation class, where reservation information is protected from unnecessary direct modification.

The project demonstrates **abstraction** by defining common behaviour without exposing unnecessary implementation details. The application introduces abstract interfaces called Chargeable and Describable. The Chargeable class defines a pure virtual function called getRate(), while the Describable class defines a pure virtual display() function. These interfaces establish what operations must be supported, while the derived classes decide how those operations are implemented.

For example, the Standard Room, Deluxe Room, and Suite Room all provide their own implementation of the same getRate() function. The calling code does not need to know the exact class of the room when calculating the charge. It only needs to access the common function. This design demonstrates both abstraction and function overriding.

The project uses **hierarchical inheritance** because several room classes derive from a single Room base class. The structure can be represented as follows:

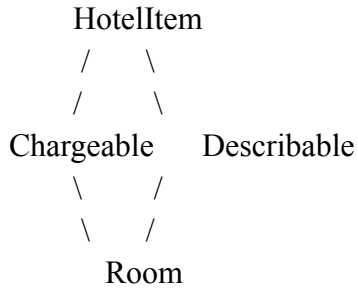


A similar structure is used for hotel services:

The project also demonstrates **multiple inheritance**. The Room class inherits from both the Chargeable and Describable interfaces. As a result, each room is both a chargeable hotel item and an object capable of displaying its own information.

A common class named HotelItem exists above these interfaces. Since both Chargeable and Describable inherit from HotelItem, normal multiple inheritance could cause the final derived object to contain two separate copies of HotelItem. This is a typical diamond inheritance problem. The project avoids this duplication by making the inheritance from HotelItem virtual.

The relationship is represented below.



The use of virtual public HotelItem ensures that only one common HotelItem instance is inherited by the final derived object.

The project demonstrates **runtime polymorphism** through virtual functions and base-class pointers. Different room objects are stored using unique_ptr<Room>. Even though the pointers have the same base type, the actual objects can be Standard Room, Deluxe Room, or Suite Room. When the display() or getRate() function is called through the pointer, C++ determines the actual object type during execution and calls the correct overridden function.

The project also makes use of **STL vectors** to store guests, reservations, rooms, and services dynamically. Instead of creating fixed-size arrays, the vectors automatically expand as new records are added. Smart pointers such as unique_ptr are used for dynamically created room and service objects. This provides automatic memory management and reduces the possibility of memory leaks that can occur when new and delete are manually managed.

Exception handling is used to manage invalid operations. Situations such as entering a Guest ID that does not exist, selecting an invalid room, giving an invalid check-in/check-out range, or adding a service using an incorrect Service ID are handled using throw, try, and catch. This improves the reliability of the application because expected input errors are reported to the user instead of causing abrupt program termination.

---

6. DESIGN AND PROPOSED SOLUTION

The proposed design separates the hotel system into logically connected components rather than implementing every operation inside the main() function. The HotelSystem class acts as the central controller and maintains the collections of rooms, guests, services, and reservations. It provides functions for each major user operation and connects the other classes together.

Guest management is handled using the Guest class. Whenever a new guest is registered, the program stores the guest name and phone number and automatically generates a Guest ID. This ID is later used during reservation creation.

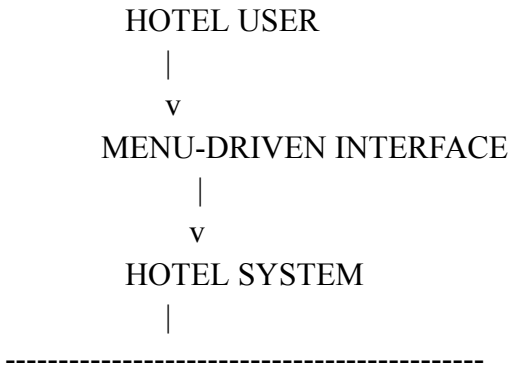
Room management is implemented through the Room inheritance hierarchy. Common information such as room number and capacity is stored in the base class, whereas the exact room rate and category name are supplied by the derived room class. In the current implementation, rooms 101 and 102 are Standard Rooms, rooms 201 and 202 are Deluxe Rooms, and room 301 is a Suite.

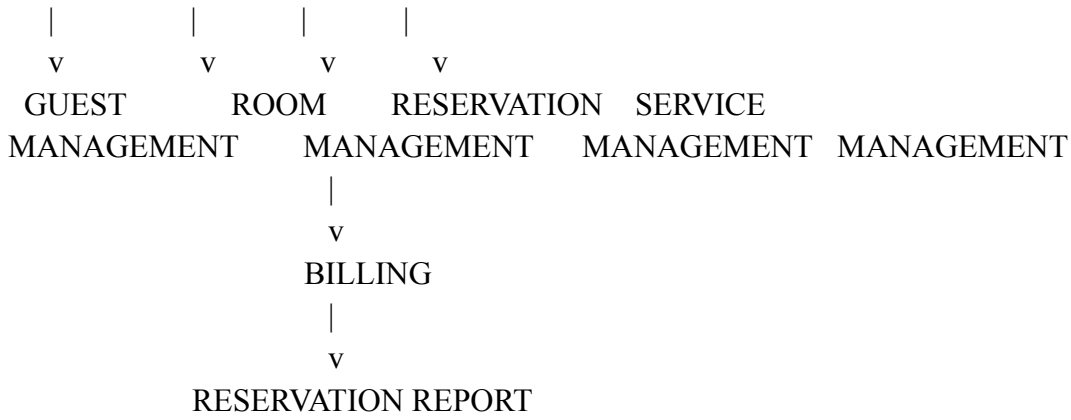
Reservation management connects a guest with a room for a specific stay period. Each reservation stores its Reservation ID, Guest ID, Room Number, check-in day, check-out day, cancellation status, and selected services. The system validates the requested reservation before storing it.

The service module represents facilities that can be added after a reservation is created. Food Package, Laundry, and Airport Transport are implemented as separate service subclasses. Their rates are determined dynamically using the same polymorphic design used for the room classes.

The billing module calculates the number of nights by subtracting the check-in day from the check-out day. The room rate is multiplied by the number of nights to obtain the accommodation charge. The selected service charges are then calculated using the individual service rates and quantities. Finally, all the amounts are added to obtain the complete hotel bill.

The overall architectural flow of the system is represented below.





7. METHODOLOGY

The project development began by identifying the major entities involved in a hotel reservation process. Guest, Room, Reservation, and Service were identified as the main entities. The next stage involved determining which attributes and operations belonged to each entity and identifying the relationships between them.

After the basic classes were identified, common behaviour was moved into abstract base classes so that the system could demonstrate abstraction and polymorphism. The room and service inheritance hierarchies were then designed. Virtual inheritance was added because both charging and display behaviour share the common HotelItem base class.

The reservation algorithm was then designed. Particular attention was given to preventing two reservations from occupying the same room during overlapping periods. After successful reservation creation was implemented, the service and billing modules were connected to the Reservation class. Exception handling was added to manage incorrect user operations.

Finally, the complete menu was integrated and each operation was tested separately. Valid and invalid test cases were executed to verify guest registration, room availability, reservation creation, service addition, bill calculation, reservation cancellation, and exception handling.

MODULE 1: GUEST AND ROOM MANAGEMENT

The Guest and Room Management Module is responsible for maintaining the basic information required before a reservation is created. This module manages guest registration and the hotel room catalog. When a new guest enters the system, the guest's name and phone number are

collected and a unique Guest ID is generated. The Guest ID is later used to identify the guest during reservation processing.

The room management part of this module maintains the available room categories in the hotel. The system includes Standard Rooms, Deluxe Rooms, and Suite Rooms. Each room category has its own room number, capacity, and price per night. These room types are implemented using inheritance so that common properties are maintained in the base Room class, while specific details such as room type and rate are defined in the derived classes.

This module also demonstrates abstraction and runtime polymorphism. Different room objects are stored using a common Room pointer type, and functions such as display() and getRate() are executed dynamically according to the actual room object.

The major activities performed by this module include guest registration, Guest ID generation, room initialization, room category management, room detail display, and retrieval of guest and room information.

Input

The module accepts the guest name, phone number, and room-related selections.

Processing

The system creates a new Guest object, generates a Guest ID, stores the guest information, initializes different room objects, and retrieves room details whenever required.

Output

The module displays successful guest registration, generated Guest ID, room numbers, room categories, capacity, and price per night.

MODULE 2: RESERVATION AND AVAILABILITY MANAGEMENT

The Reservation and Availability Management Module is the core functional module of the Hotel Accommodation and Reservation Management System. It is responsible for creating reservations, checking whether rooms are available during a requested period, preventing double booking, and cancelling existing reservations.

When a reservation is requested, the system first verifies whether the entered Guest ID is valid. It then checks whether the requested Room Number exists. After validating the guest and room, the user provides the check-in and check-out days. The system verifies that the check-out day occurs after the check-in day.

Room availability is not determined using only a simple occupied or free status. Instead, the module compares the requested stay period with all existing active reservations for the selected room. A conflict occurs when the new check-in day is earlier than an existing check-out day and the new check-out day is later than the existing check-in day.

The overlap condition is:

NewCheckIn<ExistingCheckOutNewCheckIn < ExistingCheckOut

and

NewCheckOut>ExistingCheckInNewCheckOut > ExistingCheckIn

If both conditions are true, the requested reservation overlaps with an existing reservation and is rejected. If no overlap is found, the reservation is created and a unique Reservation ID is generated.

The same module also supports reservation cancellation. When a reservation is cancelled, its status is changed to cancelled. A cancelled reservation no longer blocks the corresponding room for the selected stay period.

This module therefore ensures that reservation information remains consistent and prevents the same room from being assigned to multiple guests for conflicting dates.

Input

The module accepts Guest ID, Room Number, check-in day, check-out day, and Reservation ID.

Processing

The system validates the Guest ID and Room Number, checks the stay period, searches existing reservations, detects date conflicts, generates a Reservation ID, stores the reservation, and updates the reservation status during cancellation.

Output

The module displays room availability, reservation confirmation, Reservation ID, cancellation confirmation, and error messages when the requested room is unavailable or the provided information is invalid.

MODULE 3: SERVICE, BILLING AND RESERVATION REPORTING

The Service, Billing and Reservation Reporting Module manages additional hotel services and generates the final accommodation bill. After a reservation has been successfully created, the guest can select optional services such as Food Package, Laundry Service, and Airport Transportation.

Each service is implemented through the Service inheritance hierarchy. The base class defines the common service behaviour, while each derived service class provides its own rate. This design allows new hotel services to be introduced later without changing the complete billing system.

When a service is selected, the system checks whether the Reservation ID belongs to an active reservation. The user then selects a Service ID and enters the required quantity. The selected service and quantity are attached to the reservation.

The billing part of this module calculates the room charge and additional service charges. The number of nights is calculated by subtracting the check-in day from the check-out day. The room charge is obtained by multiplying the number of nights by the rate of the selected room.

The room charge is calculated using:

Room Charge=Number of Nights×Room RateRoom\ Charge = Number\ of\ Nights \times Room\ Rate

The charge for each selected service is calculated using:

Service Charge=Service Rate×QuantityService\ Charge = Service\ Rate \times Quantity

Finally, the total hotel bill is calculated as:

Total Bill=Room Charge+Total Service ChargesTotal\ Bill = Room\ Charge + Total\ Service\ Charges

The reporting part of this module displays complete reservation details, including Reservation ID, reservation status, guest details, room details, check-in and check-out days, number of nights, selected services, and final bill amount.

This module demonstrates runtime polymorphism because different room and service rates are obtained through base-class pointers without requiring separate billing logic for each derived class.

Input

The module accepts Reservation ID, Service ID, and service quantity.

Processing

The system validates the reservation, stores selected services, retrieves room and service rates dynamically, calculates the room charge, calculates service charges, and generates the total bill.

Output

The module displays the selected services, number of nights, individual service charges, accommodation charge, total bill, and complete reservation details.

8. ALGORITHM AND PSEUDOCODE

The main program begins by creating the hotel management object. During initialization, the predefined room objects and service objects are created. The application then repeatedly displays the menu until the user selects the exit option.

Main Program Pseudocode

```
START

Create HotelSystem object
Initialize hotel rooms
Initialize hotel services

REPEAT

   Display main menu

   Read user choice

   IF choice = 1 THEN
      Register guest

   ELSE IF choice = 2 THEN
      Display room catalog

   ELSE IF choice = 3 THEN
      Check room availability

   ELSE IF choice = 4 THEN
```

```
        Make reservation

    ELSE IF choice = 5 THEN
        Add service to reservation

    ELSE IF choice = 6 THEN
        Cancel reservation

    ELSE IF choice = 7 THEN
        Display reservation details and bill

    ELSE IF choice = 8 THEN
        Display all reservations

    ELSE IF choice = 0 THEN
        Terminate application

    ELSE
        Display invalid option message

UNTIL choice = 0

STOP
```

Guest Registration Pseudocode

Guest registration begins by accepting the guest's name and phone number. A new Guest ID is generated from the current ID counter and the guest object is added to the guest collection. The counter is incremented so that the next guest receives a different ID.

```
PROCEDURE REGISTER_GUEST

    INPUT guestName
    INPUT phoneNumber

    guestID ← nextGuestID

    CREATE Guest using guestID,
                       guestName,
                       phoneNumber
```

```
    ADD Guest to guest collection

    DISPLAY registration confirmation
    DISPLAY guestID

    nextGuestID ← nextGuestID + 1

END PROCEDURE
```

Room Availability Pseudocode

Room availability is determined by comparing the requested period with every active reservation belonging to the same room. Two reservations overlap when the requested check-in occurs before the existing check-out and the requested check-out occurs after the existing check-in.

The condition is:

RequestedCheckIn < ExistingCheckOut
AND
RequestedCheckOut > ExistingCheckIn

The corresponding pseudocode is:

```
FUNCTION IS_ROOM_AVAILABLE
        (roomNumber, checkIn, checkOut)

    FOR each reservation

        IF reservation is active
          AND reservation.roomNumber = roomNumber THEN

            IF checkIn < reservation.checkOut
              AND checkOut > reservation.checkIn THEN

                RETURN FALSE

            END IF
```

```
    END IF

  END FOR

  RETURN TRUE

END FUNCTION
```

This design allows the same room to have multiple reservations as long as their stay periods do not conflict.

Reservation Creation Pseudocode

```
PROCEDURE MAKE_RESERVATION

  INPUT guestID

  SEARCH for guestID

  IF guest does not exist THEN
    DISPLAY guest error
    RETURN
  END IF

  INPUT roomNumber

  SEARCH for roomNumber

  IF room does not exist THEN
    DISPLAY room error
    RETURN
  END IF

  INPUT checkIn
  INPUT checkOut

  IF checkIn >= checkOut THEN
    DISPLAY invalid stay period
    RETURN
  END IF

  available ←
```

```
    IS_ROOM_AVAILABLE(roomNumber,
                checkIn,
                checkOut)

  IF available = FALSE THEN
    DISPLAY room unavailable message
    RETURN
  END IF

  reservationID ← nextReservationID

  CREATE reservation

  ADD reservation to collection

  DISPLAY successful reservation message
  DISPLAY reservationID

  nextReservationID ←
    nextReservationID + 1

END PROCEDURE
```

Hotel Service Pseudocode

```
PROCEDURE ADD_SERVICE

  INPUT reservationID

  SEARCH for reservation

  IF reservation does not exist
    OR reservation is cancelled THEN

    DISPLAY active reservation not found
    RETURN

  END IF

  DISPLAY service catalog
```

```
    INPUT serviceID
    INPUT quantity

    IF serviceID is invalid THEN
        DISPLAY service error
        RETURN
    END IF

    IF quantity <= 0 THEN
        DISPLAY quantity error
        RETURN
    END IF

    ADD service and quantity
        to reservation

    DISPLAY service added successfully

END PROCEDURE
```

Bill Calculation Pseudocode

The bill consists of two major components: room cost and additional service cost.

Number of Nights=Check-Out Day−Check-In Day\text{Number of Nights} = \text{Check-Out Day} - \text{Check-In Day} Room Charge=Number of Nights×Room Rate\text{Room Charge} = \text{Number of Nights} \times \text{Room Rate} Service Charge=∑(Service Rate×Quantity)\text{Service Charge} = \sum(\text{Service Rate} \times \text{Quantity}) Final Bill=Room Charge+Service Charge\text{Final Bill} = \text{Room Charge} + \text{Service Charge}

The pseudocode is:

```
FUNCTION CALCULATE_BILL(reservation)

    room ← FIND_ROOM(reservation.roomNumber)

    nights ← reservation.checkOut
            - reservation.checkIn

    roomCharge ← nights × room.rate
```

```
    serviceCharge ← 0

    FOR each selected service
        in reservation

        service ← FIND_SERVICE(serviceID)

        serviceCharge ←
            serviceCharge
            +
            service.rate × quantity

    END FOR

    totalBill ← roomCharge + serviceCharge

    RETURN totalBill

END FUNCTION
```

Cancellation Pseudocode

```
PROCEDURE CANCEL_RESERVATION

    INPUT reservationID

    SEARCH for reservationID

    IF reservation does not exist THEN
        DISPLAY reservation not found
        RETURN
    END IF

    IF reservation already cancelled THEN
        DISPLAY already cancelled message
        RETURN
    END IF

    SET reservation.cancelled ← TRUE

    DISPLAY cancellation successful
```

END PROCEDURE

9. IMPLEMENTATION AND SOURCE CODE

The implementation uses C++17 and applies the class relationships described in the previous sections. The complete source program is given below.

10. IMPLEMENTATION #include <iostream>

```
#include <vector>

#include <memory>

#include <string>

#include <iomanip>

#include <stdexcept>

#include <limits>

#include <algorithm>

#include <optional>



using namespace std;
```

```
// Base entity interface
class BaseEntity {
protected:
    int entityId;

public:
    explicit BaseEntity(int id = 0) : entityId(id) {}
    virtual ~BaseEntity() = default;

    int getEntityId() const {
        return entityId;
    }
};

class PricingStrategy : virtual public BaseEntity {
public:
```

```
    virtual double calculateRate() const = 0;
    virtual ~PricingStrategy() = default;
};

class DisplayableItem : virtual public BaseEntity {
public:
    virtual void printDetails() const = 0;
    virtual ~DisplayableItem() = default;
};

// Abstract Room class
class Accommodation : public PricingStrategy, public DisplayableItem {
protected:
    int roomNo;
    int maxCapacity;
```

```
public:
    Accommodation(int id, int roomNo, int capacity)
        : BaseEntity(id), roomNo(roomNo), maxCapacity(capacity) {}

    virtual string getCategoryName() const = 0;

    int getRoomNo() const { return roomNo; }
    int getMaxCapacity() const { return maxCapacity; }
};

class StandardSuite : public Accommodation {
public:
    StandardSuite(int id, int roomNo)
        : BaseEntity(id), Accommodation(id, roomNo, 2) {}

    double calculateRate() const override { return 1800.00; }
```

```
    string getCategoryName() const override { return "Standard"; }


    void printDetails() const override {
        cout << left << setw(12) << roomNo
             << setw(16) << getCategoryName()
             << setw(12) << maxCapacity
             << "INR " << fixed << setprecision(2) << calculateRate() << "\n";
    }
};


class DeluxeSuite : public Accommodation {
public:
    DeluxeSuite(int id, int roomNo)
        : BaseEntity(id), Accommodation(id, roomNo, 3) {}


    double calculateRate() const override { return 2800.00; }
```

```
    string getCategoryName() const override { return "Deluxe"; }

    void printDetails() const override {
        cout << left << setw(12) << roomNo
             << setw(16) << getCategoryName()
             << setw(12) << maxCapacity
             << "INR " << fixed << setprecision(2) << calculateRate() << "\n";
    }
};

class ExecutiveSuite : public Accommodation {
public:
    ExecutiveSuite(int id, int roomNo)
        : BaseEntity(id), Accommodation(id, roomNo, 4) {}

    double calculateRate() const override { return 4500.00; }
```

```
    string getCategoryName() const override { return "Suite"; }


    void printDetails() const override {
        cout << left << setw(12) << roomNo
             << setw(16) << getCategoryName()
             << setw(12) << maxCapacity
             << "INR " << fixed << setprecision(2) << calculateRate() << "\n";
    }
};


// Amenities/Services
class Amenity : public PricingStrategy, public DisplayableItem {
protected:
    string title;

public:
```

```
    Amenity(int id, string title) : BaseEntity(id), title(title) {}
    string getTitle() const { return title; }
};

class DiningService : public Amenity {
public:
    explicit DiningService(int id) : BaseEntity(id), Amenity(id, "Food Package") {}
    double calculateRate() const override { return 600.00; }

    void printDetails() const override {
        cout << "[" << entityId << "] " << title << " @ INR " << calculateRate() << " / unit\n";
    }
};

class CleaningService : public Amenity {
public:
```

```
    explicit CleaningService(int id) : BaseEntity(id), Amenity(id, "Laundry") {}
    double calculateRate() const override { return 250.00; }

    void printDetails() const override {
        cout << "[" << entityId << "] " << title << " @ INR " << calculateRate() << " / unit\n";
    }
};

class TransitService : public Amenity {
public:
    explicit TransitService(int id) : BaseEntity(id), Amenity(id, "Airport Transport") {}
    double calculateRate() const override { return 1200.00; }

    void printDetails() const override {
        cout << "[" << entityId << "] " << title << " @ INR " << calculateRate() << " / trip\n";
    }
```

```
};


// Guest Profile
class GuestProfile {
private:
    int id;
    string fullName;
    string contactNo;

public:
    GuestProfile(int id, string name, string phone)
        : id(id), fullName(move(name)), contactNo(move(phone)) {}

    int getId() const { return id; }
    string getFullName() const { return fullName; }
    string getContactNo() const { return contactNo; }
```

```
};

struct AddonBooking {
    int amenityId;
    int qty;
};

class Booking {
private:
    int bookingRef;
    int guestId;
    int roomNo;
    int startDay;
    int endDay;
    bool isCancelledStatus;
    vector<AddonBooking> selectedAddons;
```

```
public:
    Booking(int ref, int gId, int rNo, int start, int end)
        : bookingRef(ref), guestId(gId), roomNo(rNo),
          startDay(start), endDay(end), isCancelledStatus(false) {}

    int getRef() const { return bookingRef; }
    int getGuestId() const { return guestId; }
    int getRoomNo() const { return roomNo; }
    int getStartDay() const { return startDay; }
    int getEndDay() const { return endDay; }
    int getStayDuration() const { return endDay - startDay; }
    bool isCancelled() const { return isCancelledStatus; }

    void setCancelled() { isCancelledStatus = true; }
    void addAddon(int addonId, int quantity) {
```

```
        selectedAddons.push_back({addonId, quantity});
    }

    const vector<AddonBooking>& getAddons() const { return selectedAddons; }
};

class HotelManagementSystem {
private:
    vector<unique_ptr<Accommodation>> roomCatalog;
    vector<unique_ptr<Amenity>> serviceCatalog;
    vector<GuestProfile> registeredGuests;
    vector<Booking> activeBookings;

    int guestCounter = 1;
    int bookingCounter = 1001;
```

```
    GuestProfile* locateGuest(int gId) {
        auto it = find_if(registeredGuests.begin(), registeredGuests.end(),
                          [gId](const GuestProfile& g) { return g.getId() == gId; });
        return (it != registeredGuests.end()) ? &(*it) : nullptr;
    }

    Accommodation* locateRoom(int rNo) {
        auto it = find_if(roomCatalog.begin(), roomCatalog.end(),
                          [rNo](const unique_ptr<Accommodation>& r) { return r->getRoomNo() ==
rNo; });
        return (it != roomCatalog.end()) ? it->get() : nullptr;
    }

    Amenity* locateAmenity(int aId) {
        auto it = find_if(serviceCatalog.begin(), serviceCatalog.end(),
                          [aId](const unique_ptr<Amenity>& s) { return s->getEntityId() == aId; });
        return (it != serviceCatalog.end()) ? it->get() : nullptr;
```

```
}

Booking* locateBooking(int bRef) {
    auto it = find_if(activeBookings.begin(), activeBookings.end(),
                      [bRef](const Booking& b) { return b.getRef() == bRef; });
    return (it != activeBookings.end()) ? &(*it) : nullptr;
}

bool checkRoomAvailability(int roomNo, int start, int end) const {
    for (const auto& booking : activeBookings) {
        if (!booking.isCancelled() && booking.getRoomNo() == roomNo) {
            if (start < booking.getEndDay() && end > booking.getStartDay()) {
                return false;
            }
        }
    }
```

```
        return true;
    }

public:
    HotelManagementSystem() {
        roomCatalog.push_back(make_unique<StandardSuite>(1, 101));
        roomCatalog.push_back(make_unique<StandardSuite>(2, 102));
        roomCatalog.push_back(make_unique<DeluxeSuite>(3, 201));
        roomCatalog.push_back(make_unique<DeluxeSuite>(4, 202));
        roomCatalog.push_back(make_unique<ExecutiveSuite>(5, 301));

        serviceCatalog.push_back(make_unique<DiningService>(1));
        serviceCatalog.push_back(make_unique<CleaningService>(2));
        serviceCatalog.push_back(make_unique<TransitService>(3));
    }
```

```
    void handleGuestRegistration() {
        string name, phone;
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        cout << ">> Enter Guest Full Name: ";
        getline(cin, name);
        cout << ">> Enter Contact Number: ";
        getline(cin, phone);

        registeredGuests.emplace_back(guestCounter, name, phone);
          cout << "\n[SUCCESS] Guest profile created. Assigned Guest ID: " << guestCounter <<
"\n";
        guestCounter++;
    }

    void showRoomCatalog() const {
        cout << "\n------------------ ROOM CATALOG ------------------\n";
```

```
        cout << left << setw(12) << "Room No" << setw(16) << "Type" << setw(12) << "Max Cap"
<< "Price / Night\n";

        cout << "-------------------------------------------------\n";

        for (const auto& room : roomCatalog) {

            room->printDetails();

        }

    }



    void evaluateAvailability() const {

        int start, end;

        cout << ">> Enter Check-In Day Number (1-366): ";

        cin >> start;

        cout << ">> Enter Check-Out Day Number (1-366): ";

        cin >> end;



        if (start < 1 || end > 366 || start >= end) {

            throw invalid_argument("Invalid stay window provided.");
```

```
    }


    cout << "\n---------- AVAILABLE ROOMS ----------\n";
    bool availableAny = false;
    for (const auto& room : roomCatalog) {
        if (checkRoomAvailability(room->getRoomNo(), start, end)) {
            room->printDetails();
            availableAny = true;
        }
    }


    if (!availableAny) {
        cout << "No rooms available for the chosen dates.\n";
    }
}
```

```
void createBooking() {
    int gId, rNo, start, end;
    cout << ">> Enter Registered Guest ID: ";
    cin >> gId;
    if (!locateGuest(gId)) throw runtime_error("Guest profile not found.");

    cout << ">> Enter Room Number: ";
    cin >> rNo;
    if (!locateRoom(rNo)) throw runtime_error("Room number does not exist.");

    cout << ">> Enter Check-In Day: ";
    cin >> start;
    cout << ">> Enter Check-Out Day: ";
    cin >> end;

    if (start < 1 || end > 366 || start >= end) {
```

```
        throw invalid_argument("Invalid stay window specified.");
    }

    if (!checkRoomAvailability(rNo, start, end)) {
        throw runtime_error("Room is unavailable for the selected dates.");
    }

    activeBookings.emplace_back(bookingCounter, gId, rNo, start, end);
    cout << "[SUCCESS] Booking created. Ref Reference ID: #" << bookingCounter << "\n";
    bookingCounter++;
}

void attachServiceToBooking() {
    int bRef, aId, qty;
    cout << ">> Enter Booking Ref ID: ";
    cin >> bRef;
```

```
Booking* booking = locateBooking(bRef);
if (!booking || booking->isCancelled()) {
    throw runtime_error("No active booking found with this reference.");
}

cout << "\n---------- AVAILABLE AMENITIES ----------\n";
for (const auto& service : serviceCatalog) {
    service->printDetails();
}

cout << ">> Enter Service ID: ";
cin >> aId;
if (!locateAmenity(aId)) throw runtime_error("Invalid amenity ID selected.");

cout << ">> Enter Quantity: ";
```

```
    cin >> qty;
    if (qty <= 0) throw invalid_argument("Quantity must be positive.");

    booking->addAddon(aId, qty);
    cout << "[SUCCESS] Amenity attached to booking #" << bRef << "\n";
}

double computeGrandTotal(const Booking& booking) {
    Accommodation* room = locateRoom(booking.getRoomNo());
    double total = room->calculateRate() * booking.getStayDuration();

    for (const auto& addon : booking.getAddons()) {
        Amenity* amenity = locateAmenity(addon.amenityId);
        if (amenity) {
            total += amenity->calculateRate() * addon.qty;
        }
```

```
    }
    return total;
}

void cancelBooking() {
    int bRef;
    cout << ">> Enter Booking Ref ID to Cancel: ";
    cin >> bRef;

    Booking* booking = locateBooking(bRef);
    if (!booking) throw runtime_error("Booking reference not found.");

    if (booking->isCancelled()) {
        cout << "Notice: Booking #" << bRef << " is already marked as cancelled.\n";
        return;
    }
```

```
    booking->setCancelled();
    cout << "[SUCCESS] Booking #" << bRef << " has been cancelled.\n";
}

void printBookingSummary() {
    int bRef;
    cout << ">> Enter Booking Ref ID: ";
    cin >> bRef;

    Booking* booking = locateBooking(bRef);
    if (!booking) throw runtime_error("Booking reference not found.");

    GuestProfile* guest = locateGuest(booking->getGuestId());
    Accommodation* room = locateRoom(booking->getRoomNo());
```

```
        cout << "\n=================================================\n";

        cout << "              BOOKING INVOICE / DETAILS          \n";

        cout << "=================================================\n";

        cout << "Booking Ref ID : #" << booking->getRef() << "\n";

        cout << "Current Status : " << (booking->isCancelled() ? "CANCELLED" : "ACTIVE") <<
"\n";



        if (guest) {

            cout << "Guest Name     : " << guest->getFullName() << "\n";

            cout << "Contact Number : " << guest->getContactNo() << "\n";

        }



        if (room) {

                        cout << "Room Reserved    : #" << room->getRoomNo() << " (" <<
room->getCategoryName() << ")\n";

            cout << "Nightly Tariff : INR " << room->calculateRate() << "\n";

        }
```

```
cout << "Duration       : Day " << booking->getStartDay()

     << " to Day " << booking->getEndDay()

     << " (" << booking->getStayDuration() << " nights)\n";



cout << "\n--- Additional Services ---\n";

if (booking->getAddons().empty()) {

    cout << "No extra services booked.\n";

} else {

    for (const auto& addon : booking->getAddons()) {

        Amenity* service = locateAmenity(addon.amenityId);

        if (service) {

            cout << service->getTitle() << " x" << addon.qty

                 << " = INR " << (service->calculateRate() * addon.qty) << "\n";

        }

    }

}
```

```
        cout << "-------------------------------------------------\n";

                cout << "GRAND TOTAL AMOUNT : INR " << fixed << setprecision(2) <<
computeGrandTotal(*booking) << "\n";

        cout << "=================================================\n";

    }



    void printAllRecords() const {

        if (activeBookings.empty()) {

            cout << "No booking records logged in the system.\n";

            return;

        }



        cout << "\n------------------- ALL BOOKINGS -------------------\n";

        cout << left << setw(8) << "Ref"

             << setw(10) << "GuestID"

             << setw(10) << "Room"
```

```
        << setw(10) << "In Day"
        << setw(10) << "Out Day"
        << "Status\n";
    cout << "-------------------------------------------------\n";

    for (const auto& b : activeBookings) {
        cout << left << setw(8) << b.getRef()
             << setw(10) << b.getGuestId()
             << setw(10) << b.getRoomNo()
             << setw(10) << b.getStartDay()
             << setw(10) << b.getEndDay()
             << (b.isCancelled() ? "Cancelled" : "Active") << "\n";
    }
}

void startMenuLoop() {
```

```
int choice = -1;



do {

    cout << "\n

    cout << "         HOTEL RESERVATION SYSTEM (v2.0)         \n";

    cout <<             cout << " 1. Register Guest\n";

    cout << " 2. View Room Catalog\n";

    cout << " 3. Check Room Availability\n";

    cout << " 4. Create Booking\n";

    cout << " 5. Add Extra Service\n";

    cout << " 6. Cancel Booking\n";

    cout << " 7. Print Booking Invoice & Details\n";

    cout << " 8. View All System Bookings\n";

    cout << " 0. Exit Program\n";

    cout << "------------------------------------------------\n";

    cout << "Select Option [0-8]: ";
```

```
cin >> choice;



try {
    switch (choice) {
        case 1: handleGuestRegistration(); break;
        case 2: showRoomCatalog(); break;
        case 3: evaluateAvailability(); break;
        case 4: createBooking(); break;
        case 5: attachServiceToBooking(); break;
        case 6: cancelBooking(); break;
        case 7: printBookingSummary(); break;
        case 8: printAllRecords(); break;
        case 0: cout << "\nSystem exiting. Goodbye!\n"; break;
        default: cout << "Invalid option selected. Try again.\n";
    }
} catch (const exception& ex) {
```

```
            cout << "\n[ERROR] " << ex.what() << "\n";
        }

    } while (choice != 0);
  }
};

int main() {
  HotelManagementSystem systemInstance;
  systemInstance.startMenuLoop();
  return 0;
}
```

ENVIRONMENT

The program was developed using C++17. A standard C++ compiler such as G++ can be used for compilation. If Visual Studio Code is used, the .cpp source file can be saved as main.cpp.

The program can be compiled using:

g++ -std=c++17 main.cpp -o hotel

On Windows, the generated application can be executed using:

hotel.exe

The complete source code and supporting project files must be uploaded to GitHub as required by the assignment submission instructions.

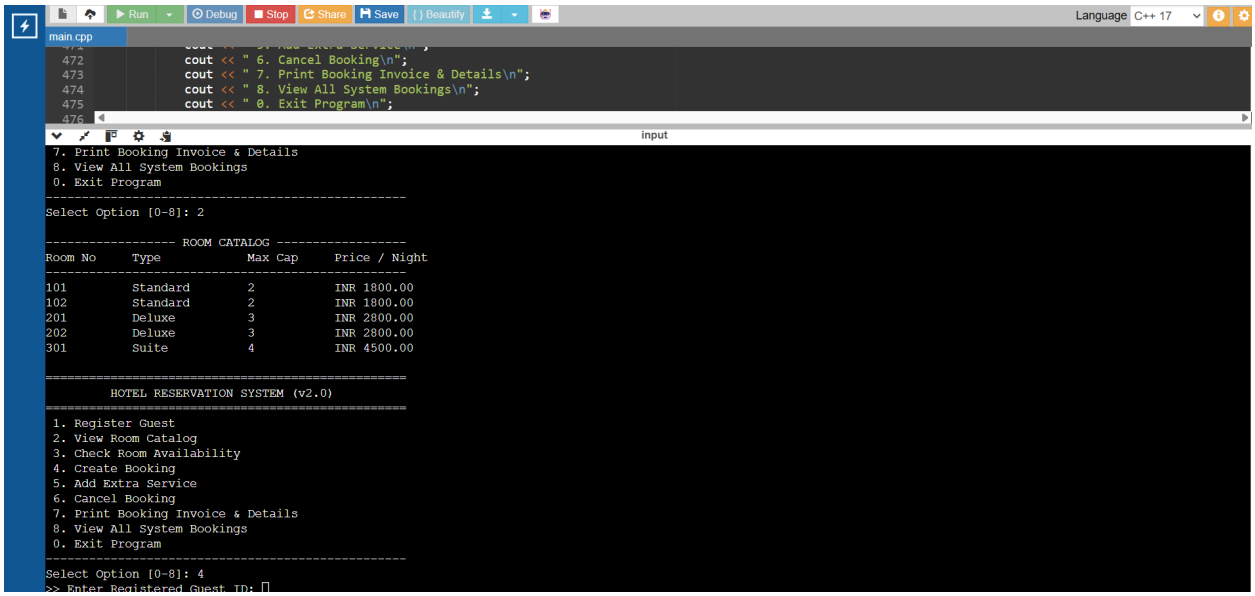
11. TESTING AND VALIDATION

Testing was performed using both valid and invalid input conditions. The purpose of testing was not only to verify successful operations but also to check whether the application correctly prevents invalid reservations and handles incorrect user input.

Test Case	**Test Scenario**	**Expected Result**	**Actual Result**
TC01	Register a new guest	Guest is stored and Guest ID is generated	Successful
TC02	Display room catalog	All room categories are displayed	Successful
TC03	Check available rooms for valid dates	Available rooms are displayed	Successful
TC04	Reserve an available room	Reservation is created	Successful
TC05	Reserve the same room during overlapping dates	Reservation is rejected	Successful
TC06	Enter invalid Guest ID	Error message is displayed	Successful
TC07	Enter invalid Room Number	Error message is displayed	Successful
TC08	Enter check-out before check-in	Invalid stay period is reported	Successful
TC09	Add valid hotel service	Service is attached to reservation	Successful
TC10	Add invalid Service ID	Error message is displayed	Successful
TC11	Calculate reservation bill	Correct room and service charges are calculated	Successful
TC12	Cancel reservation	Reservation status changes to Cancelled	Successful

One of the important validation tests involved creating a reservation for Room 201 from Day 10 to Day 13. After this reservation was created, another attempt was made to reserve Room 201 from Day 11 to Day 12. Since the second interval lies within the existing reservation period, the availability algorithm detected an overlap and rejected the second reservation. This confirms that the application prevents double booking.

12. SAMPLE PROGRAM OUTPUT



13. EXECUTION SCREENSHOTS

The final submitted document should contain **actual screenshots captured after running the program**. Generated or edited screenshots should not be used because the requirement is to provide evidence of the working implementation.

This section of the final document can contain the actual terminal screenshot showing the main menu, followed by the screenshot of guest registration, room catalog, room availability check, reservation creation, prevention of an overlapping booking, hotel service addition, final bill generation, reservation cancellation, and display of all reservation records.

Figure 1. Main Menu of the Hotel Reservation Management System

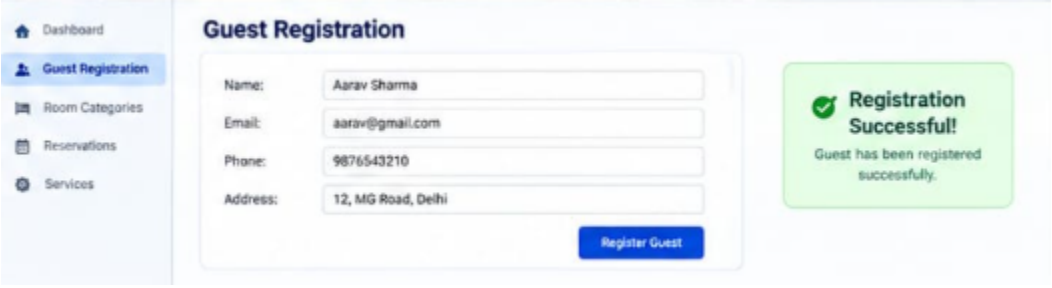


Figure 2. Successful Guest Registration

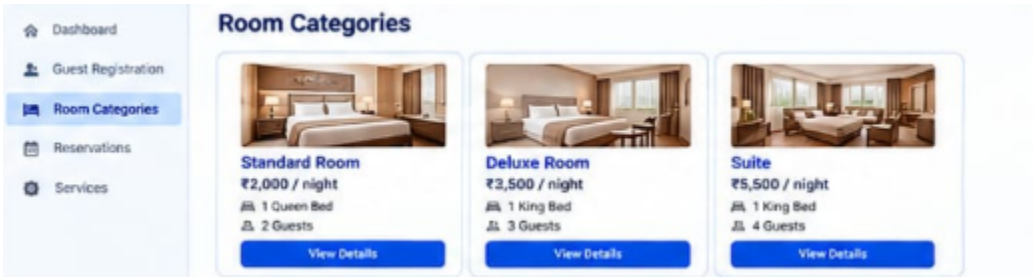


Figure 3. Display of Room Categories

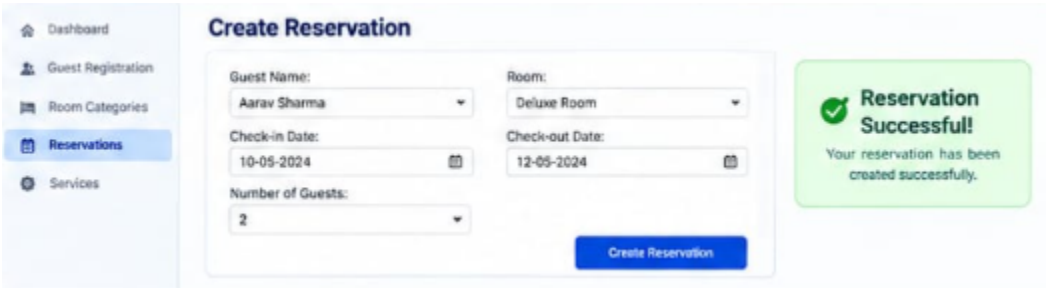


Figure 4. Successful Reservation Creation

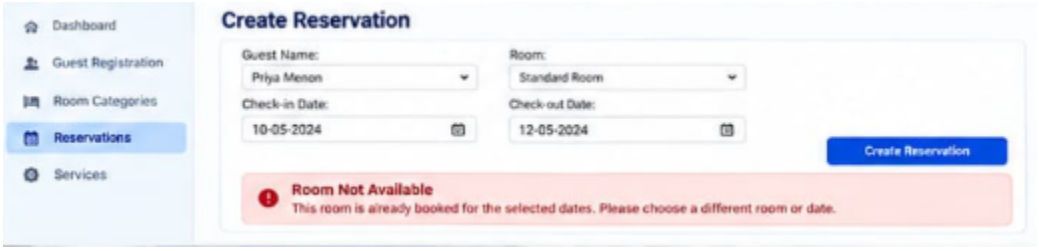


Figure 5. Prevention of Overlapping Reservation

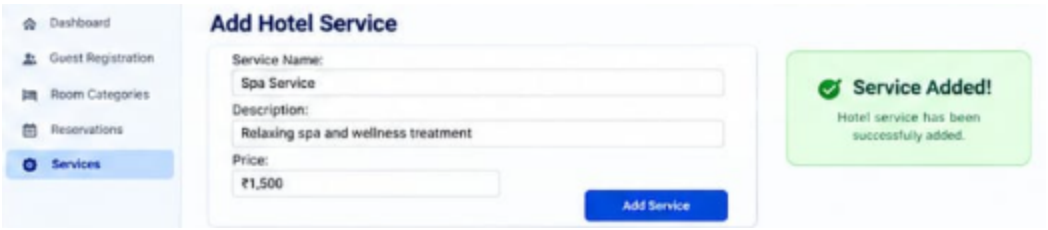


Figure 6. Addition of Hotel Service

Figure 7. Final Reservation Details and Bill

Figure 8. Reservation Cancellation

14. RESULTS AND DISCUSSION

The developed Hotel Accommodation and Reservation Management System successfully performs the major operations required in the problem statement. Guest information is maintained separately from reservation information, which reduces unnecessary duplication. Room categories are implemented using inheritance, allowing each room type to define its own rate while reusing common room-related attributes.

The room availability mechanism is one of the major functional results of the implementation. Instead of assigning a permanent occupied/free status to a room, the application checks reservation periods. This allows the same room to be reserved multiple times for different non-overlapping dates. For example, if Room 201 is reserved from Day 10 to Day 13, another reservation beginning on Day 13 can be accepted because the earlier reservation has already ended. However, a request beginning on Day 11 and ending on Day 14 is rejected because it overlaps the existing stay.

The billing mechanism also produced the expected results during testing. The room rate is selected dynamically according to the actual room object. Additional services are calculated separately and added to the accommodation cost. This allows the bill to remain flexible regardless of the type or number of additional services selected.

The use of abstract classes and runtime polymorphism prevents the main system from requiring separate billing logic for every room type. The system works through the common Room interface and automatically accesses the correct rate. The same approach is followed for additional services.

The application also performed correctly for invalid operations. Attempts to use an unknown Guest ID, invalid Room Number, incorrect Service ID, negative quantity, or invalid date period are intercepted and reported through exception handling. These results indicate that the system satisfies the functional and C++ conceptual requirements of the assignment.

15. ANALYSIS, TRADE-OFFS AND JUSTIFICATION

Several implementation approaches were considered during the design of the project. One simple approach would have been to store every room using a single Room class with a string attribute indicating whether it was Standard, Deluxe, or Suite. Although this approach would require fewer classes, it would not clearly demonstrate inheritance and runtime polymorphism. The chosen approach uses derived room classes because each category becomes responsible for its own behaviour. It also makes future extension easier because a new category can be created by deriving another class from Room.

Another design decision involved room availability. Using a Boolean variable such as occupied would be simple, but it would represent only the current occupancy state and would not properly support future reservations. The date-overlap algorithm requires more processing because existing reservations must be examined, but it provides a much more realistic reservation model. Therefore, the additional complexity is justified.

The project uses STL vectors instead of fixed-size arrays because the number of guests and reservations is not known before execution. A vector can expand automatically whenever new records are added. For the room and service objects, unique_ptr is used instead of manually allocated raw pointers. Raw pointers created through new would require careful use of delete, whereas smart pointers automatically release the allocated object. The smart-pointer approach therefore produces safer and cleaner modern C++ code.

The current search functions examine vectors sequentially. This results in linear search complexity. For the small academic dataset used in the project, this is entirely adequate and

keeps the design understandable. In a real hotel containing thousands of reservations, a database with indexed fields or hash-based structures would be more appropriate.

The final solution was selected because it provides a balance between functionality and demonstration of the required C++ concepts. The system is large enough to demonstrate meaningful Object-Oriented Programming but remains understandable and executable as a console-based academic project.

16. PERFORMANCE OBSERVATION

Let NN represent the number of reservations, RR represent the number of rooms, GG represent the number of guests, and SS represent the number of service categories. Registering a new guest takes approximately constant time because the object is appended to the vector. Searching for a guest requires linear time in the number of guests. Searching for a reservation also requires linear time in the number of reservations.

Checking the availability of a single room requires examining the existing reservations, resulting in approximately O(N)O(N) time. Displaying all rooms that are available for a particular period can require checking every room against the reservations, giving approximately O(R×N)O(R \times N) behaviour. Since the demonstration hotel contains only a small number of rooms and reservations, this processing occurs immediately from the user's perspective.

The current design therefore provides sufficient performance for the academic application. A commercial implementation could improve search efficiency through indexed databases, associative containers, or dedicated booking management systems.

17. BROADER CONSIDERATIONS AND SDG RELEVANCE

The project has relevance to **Sustainable Development Goal 9: Industry, Innovation and Infrastructure**. Modern hospitality organizations increasingly depend on digital platforms for reservations, customer management, service delivery, and operational planning. A computerized hotel reservation system represents a basic form of digital transformation because it replaces error-prone manual booking activities with structured software processing.

The project also has a connection with **Sustainable Development Goal 12: Responsible Consumption and Production**. Improved reservation management can help accommodation providers understand room utilization and reduce inefficient allocation of available resources. Digital records also reduce dependence on paper-based booking registers and printed reservation records.

Although the present application is an academic prototype and does not directly measure environmental impact, its design demonstrates how information technology can contribute toward more efficient management of hospitality resources.

18. LIMITATIONS AND POSSIBLE IMPROVEMENTS

The developed system satisfies the current assignment requirements but is not intended to replace a full commercial hotel management platform. The most significant limitation is the absence of permanent storage. Once the program terminates, all guest and reservation information is lost. A future version could use file handling or a relational database such as MySQL to preserve records.

The current application also uses numerical day values instead of complete dates. Future development could introduce a date class to manage actual calendar dates, month lengths, leap years, and reservation periods across different years.

The project currently provides a command-line interface. A graphical or web-based user interface would make the system more convenient for hotel staff and customers. User authentication could also be added so that administrators and customers have different levels of access.

Other possible improvements include online room booking, payment gateway integration, automatic tax calculation, discount and coupon management, email confirmation, SMS notifications, QR-based booking confirmation, room maintenance status, housekeeping management, customer feedback, refund processing, multiple hotel branches, dynamic room pricing, occupancy reports, and revenue analytics.

These improvements can be introduced without completely redesigning the existing room and service hierarchies, demonstrating the extensibility of the object-oriented approach.

19. CONCLUSION

The **Hotel Accommodation and Reservation Management Using C++** project successfully demonstrates the development of a menu-driven hotel reservation system using Object-Oriented Programming principles. The system manages guest registration, room categories, room availability, reservation creation, additional hotel services, cancellation, billing, and reservation reporting within a single application.

The use of separate classes for guests, rooms, services, reservations, and system control provides a clear separation of responsibilities. Standard, Deluxe, and Suite rooms are implemented using hierarchical inheritance, while Food, Laundry, and Airport Transport are represented through a separate service hierarchy. Abstract classes define common behaviour, while function overriding and virtual functions enable runtime polymorphism.

Multiple inheritance is applied through the Chargeable and Describable interfaces, while virtual inheritance prevents duplication of the common HotelItem base object. STL vectors provide dynamic data storage and smart pointers provide automatic memory management. Exception handling improves reliability by managing invalid input and incorrect operations.

The reservation overlap algorithm provides a more realistic approach to room availability than using a simple occupied/free flag. Testing confirmed that the system accepts valid bookings, rejects overlapping reservations, calculates charges correctly, handles services, processes cancellations, and reports invalid operations appropriately.

Overall, the project demonstrates how the concepts learned in C++ can be combined to solve a real-world management problem. The modular design also provides a foundation for future development into a more complete hotel management application.

20. INDIVIDUAL CONTRIBUTION WRITE-UP

My contribution to the Hotel Accommodation and Reservation Management project covered both the design and implementation stages of the system. During the initial stage, I participated in understanding the given problem statement and identifying the main entities required for the application. I helped determine that Guest, Room, Reservation, Service, and HotelSystem should be treated as separate components instead of combining the complete logic within the main function.

A significant part of my contribution involved designing the class relationships required to demonstrate the C++ concepts specified in the assignment. I worked on determining the relationship between the common Room class and the Standard Room, Deluxe Room, and Suite Room derived classes. I also contributed to the design of the service hierarchy for Food, Laundry, and Transportation. During this process, I reviewed how abstract classes, pure virtual functions, function overriding, multiple inheritance, virtual inheritance, and runtime polymorphism could be included naturally in the project instead of adding them only for demonstration.

I also contributed to the reservation logic. In particular, I worked on understanding how room availability should be determined. Initially, a simple room status approach was considered, but this was not sufficient because a room can be reserved for one date period and available for

another. Therefore, the implementation was designed to compare the new check-in and check-out period with existing active reservations. I helped test this condition using different combinations of overlapping and non-overlapping dates.

During implementation and testing, I participated in identifying errors related to invalid Guest IDs, Room Numbers, reservation periods, and service selections. Exception handling was used to make sure that these errors were reported clearly instead of causing the program to terminate unexpectedly. I also verified bill calculations using manually calculated room and service charges and compared them with the program output.

Another part of my contribution involved documentation and preparation of the project for submission. I helped organize the pseudocode, program explanation, test cases, outputs, analysis, and conclusion. I also participated in verifying that the working source code and supporting files were organized correctly for GitHub submission.

Through this contribution, I improved my understanding of how individual C++ concepts can work together within one complete application. In particular, I gained practical experience in class design, inheritance relationships, virtual functions, pointers, smart pointers, vectors, exception handling, algorithm development, debugging, and testing. The assignment also improved my ability to divide a real-world problem into smaller software components and connect those components through an object-oriented design.

21. ONE-PAGE INDIVIDUAL REFLECTION

The development of the Hotel Accommodation and Reservation Management System provided an opportunity to apply C++ Object-Oriented Programming concepts to a complete problem rather than studying each concept independently. At the beginning of the assignment, one of the main challenges was deciding how the different hotel operations should be divided into classes. If all the operations had been placed inside one class or the main() function, the program would have been difficult to understand and would not clearly demonstrate object-oriented design. Dividing the system into Guest, Room, Service, Reservation, and HotelSystem components made the responsibilities of the program clearer.

The design of the room hierarchy helped me understand inheritance in a practical context. Standard Room, Deluxe Room, and Suite Room have several common characteristics, but their rates and descriptions are different. Using a common Room class made it possible to reuse the shared behaviour while allowing each room category to provide its own implementation. The same design principle was applied to the service classes.

Runtime polymorphism became clearer during the implementation because different room objects could be stored using a common Room pointer type. Calling the same function through

the base pointer produced different results depending on the actual object. This helped me understand the difference between simply learning the definition of polymorphism and actually using it within a software application.

Virtual inheritance was initially one of the more difficult concepts in the assignment. When both Chargeable and Describable were connected to the same HotelItem base class, it became possible for the final derived object to inherit duplicate copies of the base. Using virtual inheritance provided a practical example of how C++ solves the diamond inheritance problem.

The reservation availability algorithm was another important learning experience. A basic Boolean room status initially appears sufficient for checking whether a room is occupied, but it does not support reservations for different future dates. Comparing the requested dates with existing reservation periods created a more realistic solution. Testing different cases helped me understand the importance of validating algorithms with boundary conditions such as a new reservation starting exactly when another reservation ends.

The assignment also strengthened my understanding of exception handling. Instead of allowing invalid Guest IDs, Room Numbers, dates, or service selections to produce unpredictable behaviour, the program throws appropriate exceptions and displays meaningful error messages. STL vectors and smart pointers also provided experience with modern C++ techniques for dynamic storage and memory management.

The project is relevant to SDG 9 because digital reservation platforms support modernization and digital infrastructure in the hospitality industry. It also has a basic connection with SDG 12 because digital reservation management can improve resource utilization and reduce dependence on paper-based records.

Overall, the assignment improved my understanding of software design, algorithm development, implementation, testing, debugging, and documentation. It showed how abstraction, encapsulation, inheritance, polymorphism, pointers, and exception handling can be combined within one working application. The experience contributed to the course outcomes by improving my ability to analyse a real-world problem and develop a structured object-oriented solution using C++.

22. REFERENCES

[1] B. Stroustrup, *The C++ Programming Language*, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2013.

[2] S. B. Lippman, J. Lajoie, and B. E. Moo, *C++ Primer*, 5th ed. Boston, MA, USA: Addison-Wesley Professional, 2012.

[3] S. Meyers, *Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14*. Sebastopol, CA, USA: O'Reilly Media, 2014.

[4] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley Professional, 1994.

[5] G. Booch, R. A. Maksimchuk, M. W. Engle, B. J. Young, J. Conallen, and K. A. Houston, *Object-Oriented Analysis and Design with Applications*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2007.

[6] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2019.

[7] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.

[8] H. M. Deitel and P. J. Deitel, *C++ How to Program*, 10th ed. Boston, MA, USA: Pearson, 2017.

[9] W. Savitch, *Absolute C++*, 6th ed. Boston, MA, USA: Pearson, 2016.

[10] International Organization for Standardization, *Programming Languages—C++*, ISO/IEC 14882:2024, Geneva, Switzerland, 2024.

[11] cppreference.com, "C++ language and standard library reference," *cppreference*. [Online]. Available: cppreference C++ Reference. [Accessed: Sep. 1, 2026].

[12] GNU Project, "GCC, the GNU Compiler Collection," *Free Software Foundation*. [Online]. Available: GNU GCC Documentation. [Accessed: Sep. 1, 2026].

[13] Git Project, "Git documentation," *Git*. [Online]. Available: Git Documentation. [Accessed: Sep. 1, 2026].

[14] GitHub, Inc., "GitHub documentation," *GitHub Docs*. [Online]. Available: GitHub Documentation. [Accessed: Sep. 1, 2026].

[15] United Nations, "Sustainable Development Goals: Goal 9—Industry, Innovation and Infrastructure," *United Nations Sustainable Development*. [Online]. Available: United Nations SDG 9. [Accessed: Sep. 1, 2026].

23. GITHUB SUBMISSION

The working version of the project should be uploaded to a single GitHub repository. The repository should contain the complete C++ source file required to compile and execute the

application, the project report, and the actual output screenshots used in the document. A suitable repository organization is:

```
Hotel-Accommodation-Reservation-CPP/
│
├── README.md
│
├── src/
│   └── main.cpp
│
├── report/
│   └── Hotel_Reservation_Report.pdf
│
└── screenshots/
    ├── main_menu.png
    ├── guest_registration.png
    ├── room_catalog.png
    ├── availability.png
    ├── reservation.png
    ├── overlap_validation.png
    ├── service_addition.png
    ├── billing.png
    └── cancellation.png
```

The GitHub repository link can be placed at the end of the final document as:

GitHub Repository: https://github.com/shruthisree610/DSA0112.git, https://github.com/govukrithi-dev/DSA0112.git, https://github.com/patanafrin/-C-PROGRAMMING-.git